

Chapter 2

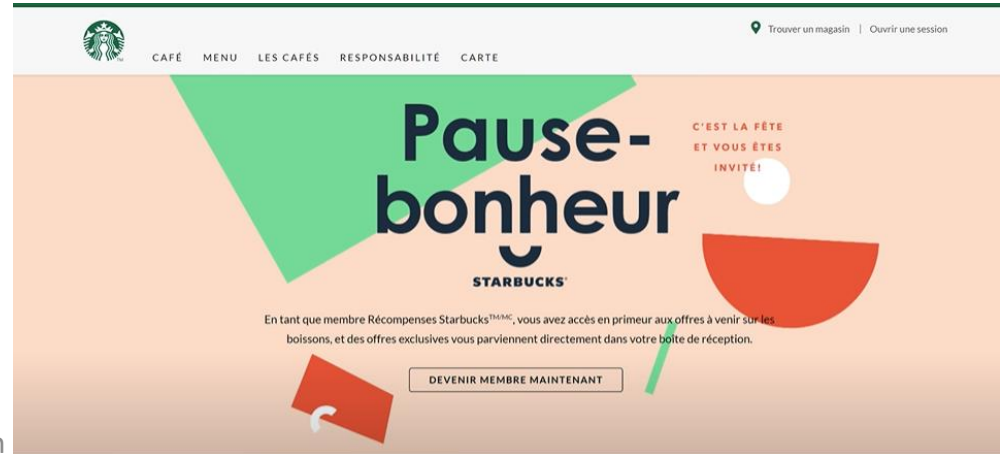
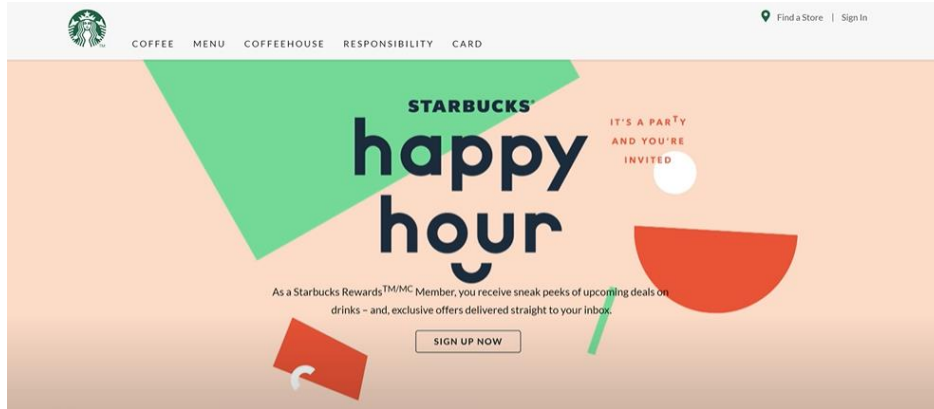
Localization and Language Computing

4 hrs

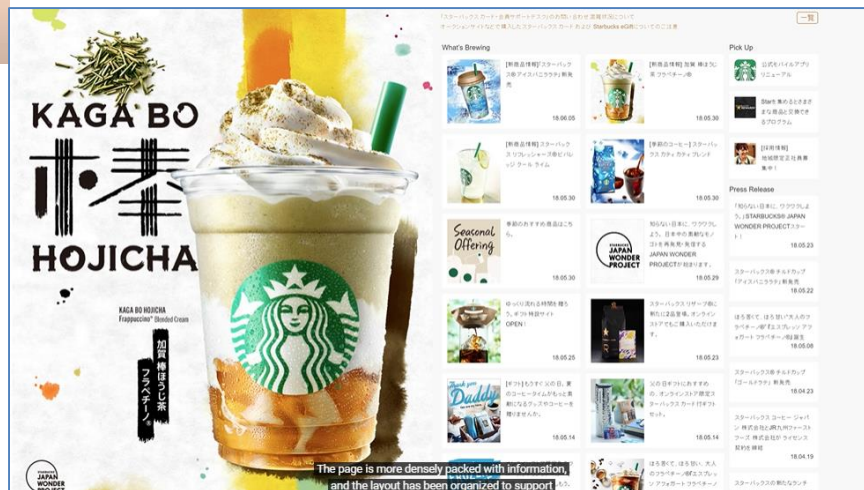
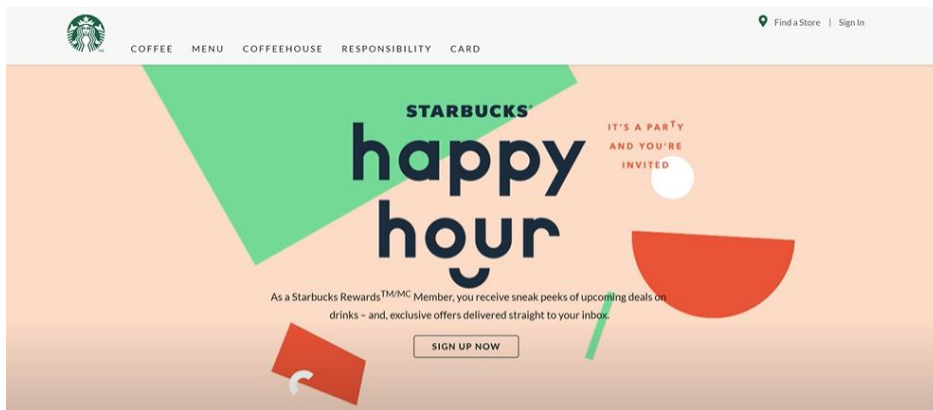
Contents:

1. Introduction to Localization
2. Key concepts of Localization
3. Unicode and Multilingual Computing

What difference can you see?



What difference can you see?



Translation Vs Localization

- **Translation** changes the language of a site, while **localization** is a process of comprehensive adaptation.

Translation
changes the language

Localization
comprehensive adaptation

2.1 Introduction to Localization

- **Localization in Natural Language Processing (NLP)** refers to the process of adapting content not just by translating it into another language, but by tailoring it to fit the cultural, linguistic, and functional expectations of the target audience in a specific locale or region.
- It as **the process of adapting, translating and customizing a product (software) for a specific market.**
- Hence this involves dealing with a specific locale or cultural conventions.
- By locale, we generally understand convents such as sort order, keyboard layout, date, time, number and currency format.
- Hence, in order to have rightly called a software being localized in the truest sense, it **should provide the local "look-and-feel" while working with it.**

Examples of Successful Localization

- **Microsoft Windows:** Localized into numerous languages, making it a globally dominant operating system.
- **Facebook:** Offers its platform in over 100 languages, contributing to its global user base of over 2 billion people.
- **Netflix:** Localizes its content and interface in many languages, allowing it to penetrate and succeed in international markets.

Why Localize your software

1. **Reach a Broader Audience:** Most people prefer using software in their **native language**.
2. **Increase Revenue:** Users are more likely to **purchase or subscribe** to software they understand and feel comfortable with.
3. **Improve User Experience:** Tailoring content, formats (dates, currencies), and cultural nuances makes users **feel valued**.
4. **Comply with Local Regulations:** Some regions **require localization** for software to be legally distributed or used. For example, the EU and certain Asian countries have **language and accessibility requirements**.
5. **Build Brand Loyalty:** Localization enhances **customer trust and brand perception**.
6. **Gain Competitive Advantage:** Offering localized versions can give you a **head start in international markets**.

Issues in Localization

- In practice, localization needs to address three main issues:
 1. Linguistic Issues
 2. Content and Cultural Issues
 3. Technical Issues

a) Linguistic Issues:

- This essentially covers the **translation of a product's user interface and documentation.**
- **Literal translations** can lead to awkward or incorrect phrases.
- **Idioms, slang, and humor** often don't translate well.
- **Context** is crucial—words can have multiple meanings.
- **Idiom:** "Break the ice" → Start a conversation in a social situation
- **Slang:** "Cool" → Great, awesome
- **Humor:** "Time flies like an arrow; fruit flies like a banana."

b) Content and Cultural Issues:

- This relates to adapting the information and functionality in products **as per the norms acceptable to the local audience.**
- Issues like the necessity of designing and developing specific software as per the locally accepted norms and regulations in terms of preference over colors, graphics, icons etc. is generally taken into consideration under content and cultural issues.
- **Content** that's fine in one culture might be **offensive or inappropriate** in another.
- **Colors, symbols, gestures**, or even images may be perceived differently in other cultures.

c) Technical Issues:

- While rendering support for local languages and content, there are certain issues that need to be addressed in localization.
- Handling bi-directional texts requires some extra effort in design and engineering.
- For example, the Arabic script as compared to Roman or Latin.
- Similarly, the fact that Far Eastern languages require twice the disk space as is required for English, where each character also needs to be considered while dealing with the localization of software in these languages.
- Different locales use different **date formats** (e.g., MM/DD/YYYY vs DD/MM/YYYY).

2.2 Key concepts of Localization

1. Translation

- Converting text from one language to another.
- Goes beyond word-for-word translation: must **preserve meaning, tone, and context**.
- Involves **UI text, error messages, menus, documents**, etc.

2. Locale

- A set of rules for a specific **language and region** (like en_US, fr_FR).
- Affects **date/time formats, number formats, currency symbols, sorting rules**, etc.

3. Cultural Adaptation

- Modifying content to suit **local customs, values, and sensitivities**.
- Includes:
 - Images and colors
 - Idioms and references
 - Humor, tone, and formality

2.2 Key concepts of Localization

4. Right-to-Left (RTL) Support

- For languages like Arabic, Hebrew, Urdu.
- UI must support **mirrored layouts**, **RTL text input**, and **correct punctuation handling**.

5. Character Encoding (Unicode)

- Supports **multilingual text** using Unicode (e.g., UTF-8).
- Prevents garbled characters and encoding conflicts.

6. Internationalization (i18n)

- Making the software ready for localization.
- Includes:
 - Separating UI text from source code
 - Using placeholders for dynamic content
 - Formatting via locale-aware libraries

2.3 Unicode

- **Unicode** is a **universal character encoding system** that assigns a unique number (code point) to **every character** in **every language**, regardless of the platform, program, or language.
- Before Unicode, different systems (like ASCII, ISO 8859) supported only **limited character sets**—causing problems when mixing languages.
- It was **designed to address the limitations of earlier encoding systems** which could not accommodate the vast number of characters used in different languages and scripts.
- While ASCII uses only 1 byte the Unicode **uses 4 bytes** to represent characters. Hence, it provides a very wide variety of encoding.
- Unicode has three types namely: **UTF-8, UTF-16, UTF-32**.
- Among them, UTF-8 is used mostly it is also the default encoding for many programming languages.

Unicode Encoding Forms

- **UTF-8:** A variable-length encoding form that uses **1 to 4 bytes** for each character. It is backward compatible with ASCII and widely used on the web.
- **UTF-16:** Uses **2 or 4 bytes for each character**. It is commonly used in software applications and operating systems. common in Windows
- **UTF-32:** Uses **4 bytes for each character**. It provides a fixed-length representation but is less efficient in terms of storage space.

Importance of Unicode

1. As it is a universal standard therefore, it **allows writing a single application for various platforms**. This means that we can develop an application once and run it on various platforms in different languages. Hence we don't have to write the code for the same application again and again. And therefore the development cost reduces.
2. Moreover, **data corruption is not possible** in it.
3. It is a **common encoding standard** for many different languages and characters.
4. We **can use it to convert from one coding scheme to another**. Since Unicode is the superset for all encoding schemes. Hence, we can convert a code into Unicode and then convert it into another coding standard.
5. It is **preferred by many coding languages**. For example, XML tools and applications use this standard only.

5. Locales

- Locale is a generic framework that is used to adjust a software for localization.
- A **Locale** is a **set of parameters** that defines the **user's language, region (country), and cultural preferences**.
- It tells the software **how to display**:
 - Language
 - Date and time formats
 - Number and currency formats
 - Sorting and collation rules
 - Paper size, measurement units, etc.

Example Locales:

Locale Code	Language	Country	Notes
en_US	English	United States	MM/DD/YYYY, \$ for currency
en_GB	English	United Kingdom	DD/MM/YYYY, £ for currency
fr_FR	French	France	Uses comma as decimal separator
hi_IN	Hindi	India	Indian currency (₹), DD-MM-YYYY
ja_JP	Japanese	Japan	Japanese script, Yen (¥), unique calendar format

End of chapter 2